AI_CONCIERGE / BACKEND_AUTH_TESTING_PROGRESS.md

AI Concierge Platform

Backend Implementation, Authentication & Testing Progress

Backend • Database • Authentication • APIs • Testing

Prepared by: Sharanya, M.Tech, PES University

Report scope: Continuation of the previous UI/UX, technical progress and implementation roadmap review

Current milestone: Authenticated backend foundation implemented and tested

This report documents the transition from the architecture and documentation phase into concrete backend implementation. It focuses on the database foundation, layered API architecture, authentication, security configuration and automated testing completed since the previous mentor review. Project objective: build a production-oriented AI Concierge that combines conversational AI, personalization, long-term memory, document intelligence, recommendations and planning through a modular architecture that can progressively introduce RAG, agents and MLOps.

PART 1 — Foundation & Data Layer

Backend skeleton, database implementation and layered architecture.

01 Executive Summary

summary/

- The backend project skeleton is operational, PostgreSQL persistence is established, and the initial database schema has been migrated.
- The API has been organized into distinct router, service, repository and schema layers.
- Eight MVP database entities are implemented using SQLAlchemy and managed through Alembic migrations.
- Authentication has progressed from design to implementation: registration, Argon2 password hashing, JWT issuance, and protected requests through a reusable current-user dependency.
- The current backend test suite contains 50 passing tests, providing a tested foundation for the first working chat flow.

02 Progress Against the Previous Roadmap

roadmap/

Foundation	Complete — FastAPI skeleton and health endpoint operational.
Database	Complete — PostgreSQL, models, Alembic, repositories, services and APIs implemented.
Authentication & APIs	Foundation complete — registration, Argon2 hashing, login, JWT and protected /users/me.
First Working Chat	Next — conversation/message flow and controlled LLM integration.

03 Backend Foundation

`foundation/`

- FastAPI application entry point with health-check endpoint.
- PostgreSQL database created and connected successfully from the application.
- SQLAlchemy engine and request-scoped database session management.
- Alembic migration environment configured and initial database migration applied.
- Environment variables used for database and authentication configuration.
- Pytest-based API testing infrastructure established using FastAPI TestClient.

04 Database & Persistence Layer

`database/`

Eight MVP SQLAlchemy models are currently implemented. The initial Alembic migration successfully created the corresponding PostgreSQL tables, establishing a reproducible schema-management process rather than relying on ad-hoc table creation.

`Users`	Identity, email, role and account timestamps
`User Preferences`	Language, response style, theme and timezone preferences
`Conversations`	Persistent conversation records linked to users
`Messages`	Conversation messages, sender, model and token metadata
`Memories`	Persistent user memory metadata; vector storage is planned separately
`Documents`	Uploaded-document metadata and storage status
`Planner Tasks`	User planning items, due dates, priority and status
`Audit Logs`	Traceable application actions and entity metadata

05 Layered Backend Architecture

architecture/

A request enters through a router, is processed by a service, and reaches the database through a repository. Pydantic schemas define the external API contract while SQLAlchemy models represent persistent entities. This structure is particularly important for the planned AI components: memory retrieval, RAG, agent orchestration and LLM calls can be introduced through services and controlled interfaces without coupling those components directly to database or HTTP implementation details.

Routers	HTTP endpoints, dependency injection, status codes and request/response handling
Schemas	Validated API request and response contracts
Services	Application logic and business rules
Repositories	Database queries and persistence operations
SQLAlchemy Models	Relational entity mapping and persistence representation
PostgreSQL	Durable structured application data

PART 2 — API & Authentication

REST API surface, authentication flow and security configuration.

06 REST API Implementation

`api/`

The backend now exposes resource-oriented APIs for the major MVP entities. The endpoints provide a stable application interface that the future React frontend can consume, and a natural boundary for later integration with the agent orchestrator and LLM service.

- User APIs: user creation and retrieval.
- User Preference APIs: creation, retrieval and update of preferences.
- Conversation APIs: creation, retrieval, listing by user and deletion.
- Message APIs: creation, retrieval, conversation-level listing and deletion.
- Memory APIs: creation, retrieval, user-level listing and deletion.
- Document APIs: creation, retrieval, user-level listing and deletion.
- Planner Task APIs: creation, retrieval, user-level listing and deletion.
- Audit Log APIs: creation and retrieval of audit records.
- Authentication APIs: registration and login.

07 Authentication Implementation

`auth/`

The registration endpoint accepts a plaintext password at the API boundary, but the password is never stored directly. The authentication service hashes the password using Argon2 through pwdlib before persisting the user. During login, the submitted password is verified against the stored hash. When the credentials are valid, the service creates a signed JWT access token containing the user's identifier as the sub claim, the user's role, and an expiration time. A reusable get_current_user dependency extracts and validates the bearer token, converts the subject into a UUID, retrieves the corresponding user from PostgreSQL, and makes that authenticated user available to protected endpoints.

Figure — current protected authentication sequence implemented in the backend.

08 Security & Configuration Improvements

security/

The authentication implementation was refined to avoid keeping security-sensitive configuration inside source code. The JWT secret, signing algorithm and access-token lifetime are now read from environment variables. The .env file remains excluded from Git so the secret is not committed to the repository.

- Argon2 password hashing through pwdlib.
- JWT expiration validation.
- Bearer-token validation through FastAPI security dependencies.
- HTTP 401 responses for invalid, malformed and expired tokens.
- Authenticated user existence checked against PostgreSQL.
- Password hashes excluded from user API responses.
- JWT secret moved from source code into environment configuration.

PART 3 — Validation & Next Steps

Testing evidence, repository structure, roadmap alignment and open questions.

09 Testing & Quality Validation

`testing/`

Automated testing has been maintained throughout the implementation rather than being deferred until the end of development. The current complete backend suite contains 50 passing tests, providing regression protection while new layers are introduced.

50

Automated tests passing — full backend test suite, spanning database/API coverage and authentication coverage for registration, duplicate registration, valid and invalid login, protected access, missing tokens, malformed tokens, invalid tokens and expired tokens.

Authentication coverage includes:

- Successful user registration.
- Duplicate-email registration rejection.
- Successful login with valid credentials.
- Wrong-password rejection.
- Non-existent-user rejection.
- Valid JWT access to /users/me.
- Missing-token rejection.
- Invalid-token rejection.
- Malformed-token rejection.
- Expired-token rejection.

The test run also reports non-blocking dependency deprecation warnings. These do not currently cause failures and can be addressed as a maintenance task after the core implementation milestone.

10 Current Backend Structure

structure/

The backend retains clear boundaries between infrastructure, persistence, application logic and HTTP interfaces. This structure is consistent with the incremental implementation approach established in the previous roadmap.

```
backend/
  app/
    main.py
    core/            database.py, security.py, dependencies.py
    models/          8 SQLAlchemy models
    repositories/     database access layer
    schemas/          Pydantic API contracts
    services/         application/business logic
    routers/          REST endpoints
  alembic/            database migrations
  alembic.ini
  requirements.txt
  tests/               API + authentication tests
```

11 Next Implementation Milestone

next/

The immediate objective is now to complete authorization and resource-ownership controls and then build the first working authenticated chat flow. The original roadmap places chat immediately after authentication and APIs, with the target sequence of user message → backend → LLM → response → stored conversation.

1. Complete authorization and ownership checks so authenticated users can access only resources they are permitted to access.
2. Refine user-creation boundaries so public APIs do not accept precomputed password hashes directly.
3. Strengthen authentication and configuration handling before broader feature integration.
4. Implement conversation and message flow as the persistence foundation for chat.
5. Introduce a controlled LLM service interface so provider-specific code is isolated.
6. Implement the first end-to-end authenticated chat interaction and persist the resulting conversation.
7. Add tests for the complete user → conversation → message → LLM response flow.
8. After chat stabilization, begin persistent memory, followed by RAG and agent/tool orchestration.

12 Alignment With the Six-Month Roadmap

timeline/

The current work is aligned with the first four-week foundation period of the previously defined six-month plan. The next major capability is the memory system, followed by RAG, agents, personalization, and productionization.

Weeks 1–4	**Backend, auth and chat foundation**	Backend/authentication foundation is implemented; chat is next.
Weeks 5–8	**Memory system**	Next major AI capability after chat.
Weeks 9–12	**RAG pipeline**	Document parsing, embeddings, Qdrant retrieval and grounded responses.
Weeks 13–16	**Agent framework**	Controlled tools and multi-step orchestration.
Weeks 17–20	**Personalization layer**	Deeper personalization and recommendation behavior.
Weeks 21–24	**Docker, CI/CD, monitoring, deployment**	Productionization and observability.

13 Mentor Discussion Points

review/

The next review should focus less on whether the backend exists and more on whether the boundaries are correct for the AI capabilities that will be added next.

- Validate the authorization and resource-ownership approach.
- Review the boundary between backend services and the future LLM/agent layer.
- Confirm the first chat MVP acceptance criteria.
- Review the choice and abstraction of the LLM provider interface.
- Confirm the order of chat → memory → RAG → agents.
- Identify measurable quality criteria for the first end-to-end flow.
- Decide which maintenance warnings should be cleaned up before the next milestone.

14 Conclusion

closing

The AI Concierge project has successfully transitioned from architecture and documentation readiness into implementation. The backend now has a working FastAPI foundation, PostgreSQL persistence, migration management, a layered service/repository architecture, resource APIs, and a functioning JWT authentication flow.

The strongest current validation point is the automated test suite: 50 tests pass after the database, API and authentication layers were integrated. The next step is therefore not to expand the system indiscriminately, but to use this stable foundation to implement a small, authenticated end-to-end chat flow. This preserves the incremental strategy established in the previous mentor report and creates a reliable base for memory, RAG, agents and eventual production deployment.

NEXT MILESTONE

Authenticated user → Conversation → Message → Controlled LLM service → Response → Persisted conversation

— End of Report —